

SevaFix

Government Application Companion — First Commit by AWS

Core idea: Prepare → Track → Repair

1. Problem

Government platforms such as myScheme already help citizens discover schemes, check basic eligibility, understand benefits, see required documents, and reach the official application portal. SevaFix should not copy that.

The harder problem starts after a person decides to apply. Forms can be confusing, documents may not match, certificates may be outdated, rules can change, and rejection messages are often difficult to understand. A citizen may know which scheme they want, but still not know whether the application they are about to submit is correct.

SevaFix focuses on this gap between finding a scheme and successfully completing the application.

2. Final idea

SevaFix is an AWS-powered application companion for government benefits. It helps a citizen build an application, checks it against the latest official rules, keeps the application history in one place, and helps repair it if it is rejected or returned.

It does not replace myScheme, CPGRAMS, or official department portals. Final submission, official status, appeals, and government decisions remain with the government system.

The product has three main parts

- Prepare — build the application, collect the right documents, and catch problems before submission.
- Track — keep the application ID, status changes, uploaded documents, and grievance history in one timeline.
- Repair — understand a rejection using official rules, fix the problem, and create a corrected version of the application.

3. Complete user journey

A. Choose the scheme

The citizen can search or select a supported scheme. myScheme and the official scheme portal can be used as starting points for scheme information. For the hackathon, SevaFix should support only one to three schemes properly instead of pretending to support thousands.

B. Build the application

SevaFix asks only the information needed for that scheme: personal details, education, income, category, bank details, address, or other scheme-specific fields. The user can save a reusable profile so the same basic information does not have to be entered again for every scheme. The system then creates a checklist of required documents and shows what is present, missing, expired, or unclear.

C. Validate before submission

Uploaded documents are read using Amazon Textract. SevaFix compares fields across the application and documents. It can flag problems such as:

- Name mismatch between the application and a certificate.
- Income entered in the form does not match the income certificate.
- A certificate belongs to the wrong financial or academic year.
- A required document is missing.
- A mandatory field is empty.
- The document does not satisfy the latest official requirement.

Instead of giving a fake 'success probability', the app shows a simple readiness result such as '12 of 13 checks passed' and clearly explains the one remaining issue.

D. Submit on the official portal

When all checks are complete, SevaFix shows 'Application Ready' and sends the user to the official government portal. For the hackathon version, SevaFix should not automate submission unless an official public API specifically allows it.

E. Track the application

After submission, the user saves the official Application ID in SevaFix. If the government system exposes a public status API or status page that can legally be used, SevaFix can read that public status. If OTP, login, CAPTCHA, or another protected step is required, SevaFix should not bypass it.

The user can instead update the status manually or upload a screenshot/message from the official portal. Textract reads it and updates the timeline.

Example timeline: Application prepared → Submitted → Under verification → Returned for correction → Corrected → Resubmitted → Approved.

F. Diagnose rejection or grievance

If the application is rejected or returned, the user uploads the rejection message, PDF, or screenshot. SevaFix extracts the reason and searches only trusted official sources for the relevant rule.

The answer should say what likely failed, what official rule applies, what the citizen should change, and where the official next step is. Every important explanation must show the source it came from.

G. Repair and rebuild

The user presses 'Fix my application'. SevaFix reopens the saved application, highlights the failed field or document, accepts the corrected information, reruns the checks, and creates a new version.

Application V1 remains available for history. Application V2 contains the corrected information. This makes the full journey understandable and auditable.

4. Where the data comes from

SevaFix should use only public and official information. It should never treat a random blog, YouTube video, or third-party article as the final authority for eligibility or application rules.

Trusted source order

- Official ministry or department notification, circular, guideline, or scheme document.
- Official scheme or department portal.
- eGazette for government notifications and amendments.
- India Code when an Act, rule, regulation, order, or notification is relevant.
- myScheme for scheme discovery, eligibility summaries, required documents, and official application links.
- data.gov.in when a suitable official dataset or API exists.
- DBT Bharat for relevant Direct Benefit Transfer scheme information.
- CPGRAMS for official grievance submission and grievance status.

For every supported scheme, SevaFix keeps a Source Registry containing the official pages and documents that define the current rules.

5. Keeping policies and rules current

This is one of the most important parts of the product. Government rules can change, so SevaFix must not keep one static copy of a policy forever.

Amazon EventBridge can schedule regular checks of registered official sources. AWS Lambda compares the latest page or document with the previously stored version. If something changes, the new file is stored in Amazon S3 and the system identifies what changed.

A policy change should not silently overwrite the old rule. SevaFix stores versions with their effective dates. For example:

- Policy V1 — income limit ₹2.5 lakh — valid until a given effective date.
- Policy V2 — income limit ₹3 lakh — valid from the new effective date onward.

This matters because an application submitted under an older rule should be checked against the rule that applied on the date of submission, not simply against today's rule.

When a source changes but has not yet been safely processed, the user should see a warning such as 'Official source recently changed — verify before submission' rather than an overconfident answer.

6. Trust and safety

Citizens should not have to blindly trust SevaFix. They should be able to verify it.

- Every important rule is tied to an official source.
- Show the source document or portal used for the answer.
- Show when the source was last checked.
- Use deterministic validation for clear rules; do not let the LLM invent eligibility conditions.
- If the system is unsure, say so and direct the user to the official authority.
- Do not bypass government login, OTP, CAPTCHA, or protected systems.
- Do not claim that SevaFix approves applications. Only the government authority can do that.

Personal documents should be encrypted in storage, access should be restricted through authentication, and users should be able to remove their stored application data. Sensitive documents should be kept only as long as needed.

7. AWS architecture

The AWS services should support the product rather than being added only for the hackathon.

- Amazon S3 — official policy documents and user-uploaded application documents.
- Amazon Textract — extract text and fields from certificates, forms, rejection PDFs, and screenshots.
- Amazon Bedrock + Bedrock Knowledge Bases — explain official rules in simple language using source-grounded retrieval.
- Amazon DynamoDB — user profiles, scheme metadata, application versions, timelines, source metadata, and check results.
- AWS Lambda — validation logic, source comparison, status processing, and backend actions.
- Amazon EventBridge — scheduled monitoring of official policy sources.
- AWS Step Functions — manage multi-step flows such as validation, policy-update review, and repair.

- Amazon Cognito — secure user login.
- Amazon SNS — optional alerts when an application status changes or action is needed.
- Amazon CloudWatch — logs, errors, monitoring, and auditability.

Important design rule: Bedrock explains and summarizes, but hard eligibility or validation rules should come from structured official requirements whenever possible.

8. First Commit MVP

The hackathon version should be deliberately small and complete. One fully working scheme is better than a chatbot that claims to understand every scheme in India.

Minimum features

- One supported government scheme with official public source documents.
- Application builder for that scheme.
- Document upload and Textract extraction.
- At least four useful validation checks.
- Source-backed explanation of each failed check.
- Application readiness checklist.
- Application ID and status timeline.
- Upload a rejection and diagnose it.
- 'Fix my application' flow that creates Application V2.
- Simple policy versioning or simulated policy-change detection.
- Clear link back to the official application or grievance portal.

9. Three-minute demo flow

- Choose the supported scholarship/benefit scheme.
- Fill basic details and upload sample documents.
- Textract reads the documents.
- SevaFix finds a mismatch before submission and shows the official rule behind the warning.
- Correct the mismatch and reach 'Application Ready'.
- Show that submission continues on the official portal.
- Open a saved application and upload a sample rejection message.
- SevaFix explains the rejection using the official source.
- Press 'Fix my application', replace the bad document, and create Application V2.
- Finish on the timeline showing Prepare → Submit → Reject → Repair → Resubmit.

10. Why this is different

myScheme mainly helps people discover schemes and understand how to apply. SevaFix focuses on successful completion of the application journey.

The strongest differentiators are:

- Pre-submission document and consistency checks.
- Rules tied to official, versioned sources.
- Application history and status timeline.
- Rejection diagnosis instead of only grievance filing.
- Repairing and rebuilding the same application instead of starting again.
- The same engine could later be integrated into myScheme, UMANG, CPGRAMS, or individual department portals rather than competing with them.

11. Main risks and how we handle them

- Different portals work differently — support a small number first and add connectors only when public interfaces exist.
- Policies can change — continuously monitor official sources and keep dated policy versions.
- AI can hallucinate — ground answers in official documents and show the source.
- Tracking may require login or OTP — never bypass it; let the citizen authenticate officially or upload the latest status.
- Personal documents are sensitive — minimize storage, encrypt data, restrict access, and allow deletion.
- A validation check is not an approval guarantee — clearly state that the government authority makes the final decision.

12. Final pitch

SevaFix is an AWS-powered companion for government-benefit applications. Instead of replacing myScheme or government portals, it handles the difficult steps in between: it helps citizens prepare an application correctly, checks documents against the latest official rules, keeps the application journey in one place, and if something goes wrong, explains the rejection and rebuilds the corrected application.

The entire product can be remembered in three words: Prepare → Track → Repair.